

B+ tree search algorithms are used to search data efficiently and can be used as follows:

1. Call the binary search method on the nodes of the B+ tree.
2. If the search parameters are matched then:

The accurate result is returned and displayed to the user.

Else, if the node being searched is the current and the exact key is not found:

Display the statement “Recordset cannot be found!”

However, though such B+ tree algorithms may be efficient, the database can still perform poorly during random access. RocksDB can resolve this problem because it uses a data structure called Log Structured Merge Tree (LSMT). This has a hybrid kind of data structure and combines various components of data structures to store/retrieve data. It is generally assumed that sequential access is faster than random access, whether it's magnetic storage or an SSD. As a B+ tree allows both random and sequential access, during the random access operations, the performance of the database can be affected, even when used with SSDs. That's why LSMT is used by RocksDB to overcome this problem.

In LSMT, data is structured as logs at various levels and stored in the

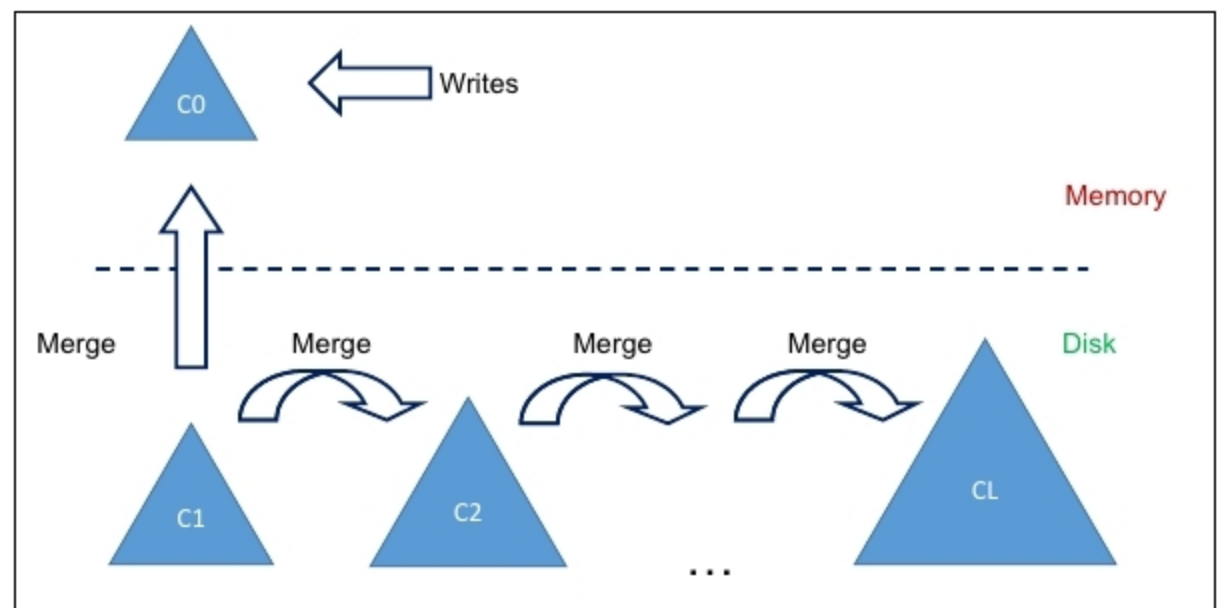


Figure 2: LSMT data structure

memory and disk. A minimum two-level LSMT uses two logs, one for the memory and another for the disk storage (Flash/ SSD). For example, C0 is the top-level log and stays in the memory. Whereas C1 is a segment of C0 but is stored in the disk, and the elements are inserted in the C0 log. When the size of the log exceeds the memory, the log is merged with the C1 log on the disk. Similarly, more than two levels can have more merges and logs, like C2, C3, C4... CL logs (L = total number of levels). As the latency of the main memory is less than the disk, storing recently updated logs in the memory can improve the performance of the database. This method of storing data is relevant to sequential access and has better IO throughput.

Using RocksDB with LSMT improves the performance of the embedded database, but a good DBMS also helps automate and maintain the database efficiently. MySQL is one such DBMS that can perform various types of operations with the Structured Query Language (SQL), which programmers use to create, modify and extract data from the relational database and also control user access to the database. It is an open source relational database management system (RDBMS), which organises data into one or more data tables, in which the data types may be related to each other and these relations help structure the data.

SQL language uses keywords, identifiers, clauses, expressions, predicates, queries, statements and

The pros and cons of MyRocks

Pros	Cons
MyRocks integrates RocksDB with MySQL and makes it possible to use SQL for the embedded database.	MyRocks doesn't make use of additional memory like InnoDB, and it doesn't benefit from increasing the memory size.
MyRocks is 2x smaller than InnoDB (compressed) and 3.5x smaller than InnoDB (uncompressed).	The read amplification of MyRocks can be improved with additional memory but still remains greater than InnoDB.
MyRocks does not require a lot of memory and performs the IO operations constantly, while using most of the CPU resources.	MyRocks uses a lot of CPU no matter how much memory is allocated to it, and that's why its performance is limited to the CPU.
MyRocks is a write-optimised engine and reduces the write amplification to a greater extent.	